

Einführung

Der Umgang mit Jupyter Notebooks

Code ausführen

In diesem Notebook kannst du Python-Code direkt ausführen. Entsprechende Code-Blöcke sind grau hinterlegt.

- ☞ Führe den folgenden Block aus, indem du das Feld auswählst und entweder oben in der Leiste auf ► klickst oder ⌘ + ↵ drückst.
- ☞ Mit **Strg + ↵** kannst du einen Block ausführen ohne zum nächsten Block weiterzuspringen.

```
In [ ]: # Das ist ein ausführbares Feld.  
# Zeilen, die mit dem Symbol # beginnen, sind Kommentare und werden vom Compiler ignoriert.  
  
print("Herzlich Willkommen im Lehr-Lern-Labor Informatik am KIT!")
```

Den Python-Code könntest du einfach in ein Python-File kopieren und von dort ausführen. Jupyter Notebooks ermöglichen es jedoch, einzelne Code-Blöcke getrennt von anderen auszuführen. Das hat den Vorteil, dass du dich auf einzelne Code-Abschnitte konzentrieren kannst, kann aber unter Umständen auch dazu führen, dass sich dein Code unerwartet verhält, weil der Code nicht von oben nach unten ausgeführt wurde.

- ☞ Die Zahlen in den eckigen Klammern links vom Block verraten dir, dass und in welcher Reihenfolge der Code ausgeführt wurde.

Kernel neu starten

Wenn sich dein Programm unerwartet verhält, kannst du den *Kernel* neu starten. Dadurch wird der aktuelle Zustand des bereits ausgeführten Codes verworfen und du kannst bei der Ausführung von Code wieder bei null beginnen.

Um den Kernel neu zu starten hast du verschiedene Möglichkeiten:

- **Klicke im Menü auf [Kernel > Restart Kernel and Run up to Selected Cell](#).**
Dadurch wird der Kernel neu gestartet und der gesamte Code bis zum ausgewählten Block wieder ausgeführt.
- **Klicke im Menü auf [Kernel > Restart Kernel and Clear Output of All Cells](#).**
Dadurch wird der Kernel neu gestartet und die bisherigen Ausgaben werden gelöscht. Den Code musst du dann selbst wieder ausführen.
- **Klicke oben in der Leiste auf [↻](#).**
Dadurch wird der Kernel neu gestartet, die bisherigen Ausgaben bleiben aber erhalten. Den Code musst du dann selbst wieder ausführen.
- **Klicke oben in der Leiste auf [▶▶](#).**
Dadurch wird der Kernel neu gestartet und der gesamte Code von oben nach unten ausgeführt - auch unterhalb des aktuellen Blocks.

Welche Variante du verwendest, bleibt dir überlassen und hängt unter Umständen von der jeweiligen Situation ab. Wichtig ist, dass du weißt, wie man den Kernel neu startet und was dadurch passiert.

Speichern

Um deine Änderungen nicht zu verlieren, solltest du das Notebook regelmäßig speichern. Dabei wird nicht nur dein Code gespeichert sondern auch die zugehörigen Ausgaben. Der Zustand des Kernels wird allerdings nicht gespeichert, sodass du den Code das nächste Mal am besten wieder von ganz oben aus ausführst.

Das Notebook kannst du genauso wie in anderen Editoren speichern, z.B. durch Klicken auf [💾](#), durch Drücken von **Strg + S** oder im Menü über **File > Save Notebook**.

Navigation & Übersichtlichkeit

Inhaltsverzeichnis

Im Menü kannst du über **View > Table of Contents** (oder alternativ mit **Strg + ⌘ + K**) das Inhaltsverzeichnis anzeigen lassen. Darüber kannst du schneller im gesamten Dokument navigieren und hast einen Überblick über die gesamte Struktur des Notebooks.

- ☞ Wenn du im Inhaltsverzeichnis links oben auf [=](#) klickst, kannst du dir die Überschriften nummeriert anzeigen lassen.

Zeilennummern

Im Menü kannst du über **View > Show Line Numbers** Zeilennummern anzeigen lassen. Das hilft dabei, über spezielle Stellen im Code zu sprechen.

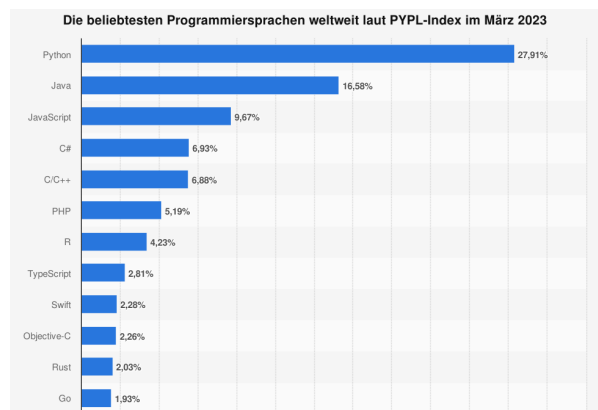
Inhalte einklappen

Wenn du mit der Maus über eine Überschrift fährst, erscheint links ein kleiner Pfeil. Damit kannst du ganze Abschnitte einklappen, wenn du sie gerade nicht benötigst und dann auch wieder ausklappen. Beispielweise könntest du das gesamte Kapitel über den Umgang mit Jupyter Notebook einklappen, um das Notebook kürzer zu machen und dich besser auf die folgenden Abschnitte konzentrieren zu können - aber natürlich nur, wenn dir jetzt klar ist, wie du mit einem Jupyter Notebook umgehst.

🗨️ *Übrigens werden eingeklappte Inhalte auch im Inhaltsverzeichnis entsprechend angezeigt. Du kannst daher auch über das Inhaltsverzeichnis Inhalte ein- und auch wieder ausklappen.*

Warum Python?

Python ist eine Programmiersprache, die in den letzten Jahren immer beliebter wurde. Python lässt sich aufgrund der einfachen Syntax schnell erlernen und bietet insbesondere zu aktuellen Themen der künstlichen Intelligenz wie beispielsweise *tiefen neuronalen Netzen* oder *Sprachverarbeitung* (Natural Language Processing) eine breite Auswahl an Bibliotheken (d.h. Code den man für seine eigenen Projekte benutzen kann ohne alles selbst schreiben zu müssen). Laut PYPL-Index ("Popularity of Programming Language Index") ist Python momentan sogar die beliebteste Programmiersprache weltweit.



Auch für unsere Anwendungsfälle eignet sich Python, weshalb du mit den Grundlagen der Sprache vertraut sein solltest.

Syntax

Einrückungen

Python arbeitet mit Einrückungen, um die Struktur des Programms sichtbar zu machen. Aus anderen Programmiersprachen wie beispielsweise *C#* oder *Java* kennst du vielleicht die geschweiften Klammern, die dort stattdessen verwendet werden. Das bedeutet aber auch, dass du deinen Code nicht beliebig einrücken darfst. Achte also im Folgenden besonders darauf, welcher Code eingerückt werden muss.

```
In [ ]: # Hier wurde richtig eingerückt.
good = "Hallo!"
print(good)

# Diese Einrückung ist falsch
bad = "Hi!"
    print(bad)
```

Variablen & Datentypen

In Python ist es denkbar einfach, eine Variable zu definieren:

```
name = wert
```

Der `name` einer Variablen kann frei gewählt werden, sollte aber aussagekräftig sein - das verbessert die Lesbarkeit des Codes.

Und auch der `wert` einer Variable kann frei gewählt werden. Im Gegensatz zu anderen Programmiersprachen musst du den Datentyp einer Variablen (z.B. Zahl, Wahrheitswert oder Zeichenkette) nicht im Vorhinein festlegen. Stattdessen wird während der Ausführung des Programms entschieden, welchen Datentyp die Variable haben sollte.

```
zahl = 42
dezimalzahl = 4.2
wahr = True
falsch = False
string = "Eine Zeichenkette in Anführungszeichen"
```

Erfahrung sammeln

Variablen

Du kannst Variablen deklarieren, ihnen Werte zuweisen, sie miteinander kombinieren und sie ausgeben lassen. Dabei helfen dir die folgenden Beispiele.

Zahlen

```
x = 42          # deklarieren und initialisieren (d.h. den ersten Wert zuweisen)
x = 2          # Wert zuweisen

# Rechnen
a = x + 1      # Addition
b = x - 2      # Subtraktion
c = x * 3      # Multiplikation
d = x / 4      # Division
e = x ** 5     # Potenz
f = x // 6     # Ganzzahlige Division
g = x % 7      # Modulo (Rest der ganzzahligen Division)

# Kurzschreibweisen
x += 2         # x = x + 2
x -= 2         # x = x - 2
x *= 2         # x = x * 2
x /= 2         # x = x / 2
x **= 2        # x = x ** 2

# Zahlen ausgeben
print(x)
```

Zeichenketten

```
hello = "Hallo"      # Zeichenketten, also Strings, deklarieren
world = 'Welt'       # "" und '' können beide verwendet werden

greeting = hello + world # Konkatenation (d.h. Zeichenketten aneinanderhängen)
length = len(greeting)  # Länge einer Zeichenkette
a = greeting[0]         # Erstes Zeichen (die Nummerierung beginnt bei 0)
b = greeting[-1]        # Letztes Zeichen (negative Nummern zählen von hinten)
c = greeting[3:6]       # Teil der Zeichenkette (einschließlich Zeichen Nummer 3, ausschließlich
Zeichen Nummer 6)
d = greeting[:5]        # d = greeting[0:5] (Erster Teil der Zeichenkette, die ersten 5 Zeichen)
e = greeting[5:]        # e = greeting[5:-1] (Letzter Teil der Zeichenkette, ab Zeichen Nummer 5)

# Zeichenketten ausgeben
print(greeting)
print(hello, world)
print(hello, "an die gesamte", world, "!")

# 🗨 Zeichenketten besser formatieren (Weitere Infos unter https://docs.python.org/3/tutorial/inputoutput.html)
print(f"{hello} an die gesamte {world}!")
```

Wahrheitswerte

```
wahr = True
falsch = False

# Boolesche Operatoren
a = not wahr      # Verneinung
b = wahr and falsch # Und
c = wahr or falsch  # Oder

# Vergleichen
d = (wahr == falsch) # Gleichheit
e = (wahr != falsch) # Ungleichheit
f = (1 < 2)          # Kleiner
g = (1 <= 2)         # Kleiner oder gleich
h = (1 > 2)          # Größer
i = (1 >= 2)         # Größer oder gleich
j = (0 <= x < 42)    # Zahlenbereich
```

Funktionen

Um Code nicht mehrfach schreiben zu müssen, verwendet man Funktionen, die an beliebigen Stellen im Programm aufgerufen werden können.

Funktionen besitzen in Python den folgenden Aufbau:

```
# Funktion definieren
def funktion(parameterA, parameterB):
    tuEtwas()
    return rückgabe
```

```
# Funktion aufrufen
funktion(wertA, wertB)
```

Du kannst den Namen der Funktion (fast) frei wählen und musst bei der Definition einer Funktion nicht festlegen, ob ein Wert zurückgegeben wird oder nicht. Falls du keinen Wert zurückgeben möchtest, lass die `return` -Zeile einfach weg.

Kontrollstrukturen

Bedingungen (if / else)

Bedingungen besitzen in Python den folgenden Aufbau:

```
if (bedingungA):
    tuEtwas()
elif (bedingungB):
    tuEtwasAnderes()
else:
    tuSonstEtwas()
```

Dabei brauchst du weder den `elif` - noch den `else` -Block. Und auch auf die Klammern um die Bedingung kannst du verzichten.

Schleifen

while-Schleife

Der Code in der while-Schleife wird solange ausgeführt, bis die Bedingung der while-Schleife nicht mehr erfüllt ist. Wenn die Bedingung nie auf *falsch* gesetzt wird, dann wird die while-Schleife unendlich oft ausgeführt und dein Programm wird nie regulär beendet.

While-Schleifen besitzen in Python den folgenden Aufbau:

```
while bedingung:
    tuEtwas()
```

for-Schleife

Möchtest du über einen Zahlenbereich (ganze Zahlen), eine Zeichenkette oder allgemein über ein iterierbares Objekt iterieren, so eignet sich die for-Schleife dazu.

For-Schleifen besitzen in Python den folgenden Aufbau:

```
for element in iterierbaresObjekt:
    tuEtwas(element)
```

Häufig möchte man dabei mit einer Zählvariablen arbeiten. Dabei hilft die `range` -Funktion.

```
for zähler in range(start, stop, schritt):
    tuEtwas(zähler)
```

```
range(2, 42, 2)    # Zähle von 2 bis ausschließlich 42 in Zweier-Schritten
range(1, 30)      # Zähle von 1 bis ausschließlich 30 (in Einser-Schritten)
range(5)          # Zähle von 0 bis ausschließlich 5 (also 5 Zahlen)
```

Daten

Tupel

In Tupeln kannst du mehrere Informationen speichern. Einmal definiert, können die Werte in einem Tupel nicht mehr verändert werden.

```
# Tuple definieren
tupel = (wertA, wertB, wertC)

# Tuple auslesen
tupel[0]    # erstes Element
```

```
tupel[-1]    # letztes Element

len(tupel)   # Länge des Tupel
```

Listen

Du kannst mit Hilfe von Listen mehrere Elemente speichern, so wie du es oben bei Tupeln gesehen hast. Allerdings kannst du die Elemente von Listen verändern, löschen und neue Elemente hinzufügen.

```
# Liste definieren und erweitern
list = [wertA, wertB, wertC]    # Liste definieren
otherList = [wert1, wert2, wert3]

list.append(wertD)               # Element hinten hinzufügen
list = list + wertE             # Element zu Liste hinzufügen

newList = list + otherList      # Zwei Listen aneinanderhängen
zip(list, otherList)            # Hängt Paare aus Elementen aus beiden Listen aneinander:
[[wertA, wert1], ...]

# Liste auslesen
list[0]                         # erstes Element
list[-1]                       # letztes Element

len(list)                      # Länge der Liste
```

```
erstesElement = list.pop()      # Erstes Element entfernen und zurückgeben
zweitesElement = list.pop(1)    # Zweites Element entfernen und zurückgeben
```

Bei Listen wird in der Regel nur die *Referenz* kopiert, d.h. wenn man in der "Kopie" etwas verändert, wird auch das Original verändert. Um das zu verhindern, muss man die Liste gezielt kopieren.

```
list[0] = neuerWert             # Element verändern
newList = list                  # Referenz kopieren (newList[0] = neuerWert ändert auch list[0])
newList = list[:]               # Liste kopieren
```

Dictionaries

Abgesehen von Listen, Strings und Tupeln gibt es in Python noch einen weiteren wichtigen sequentiellen Datentyp: *Dictionaries* (in anderen Programmiersprachen auch als *Maps* oder *Hashs* bezeichnet). Sie ermöglichen den Zugriff auf Werte, die sogenannten *values*, über den Namen, den sogenannten *key*.

```
# Dictionary definieren und erweitern
dictionary = {keyA : valueA, keyB : valueB, keyC : valueC} # Dictionary definieren
dictionary[keyD] = valueD                                # Neues Wertepaar hinzufügen bzw. Eintrag
                                                            überschreiben
newDictionary = dict(zip(listA, listB))                   # Neues Dictionary aus zwei Listen
                                                            zusammenfügen

# Dictionary auslesen
dictionary[keyA]                                          # Zu keyA gehörigen Wert auslesen
dictionary.keys()                                        # Liste aller Namen
dictionary.values()                                      # Liste aller Werte
```

Klassen

Objekte einer Klasse können nicht nur Daten, sogenannte *Attribute* enthalten sondern auch Funktionalitäten bereitstellen.

```
# Klasse definieren
class Klasse:
    # Konstruktor, wird beim Erzeugen einer Instanz aufgerufen
    def __init__(self, parameterA, parameterB):
        self.attributA = parameterA
        self.attributB = parameterB

    # Funktion, beachte die Verwendung von 'self' als ersten Parameter
    def funktion(self, parameterA):
        tuEtwas()

    def getAttributA(self):
        return self.attributA

    def setAttributB(self, neuerWert):
        self.attributB = neuerWert

# Klasse verwenden
instanz = Klasse(wertA, wertB) # Klasse instanziiieren, d.h. ein Objekt der Klasse erzeugen
instanz.funktion(argument)     # Funktion aufrufen
instanz.setAttributB(neuerWert)
```

Import

Möchtest du auf Code anderer Dateien oder auf deine lokalen Bibliotheken zugreifen, musst du sie vorher in dein Programm importieren. Danach kannst du auf die Funktionen des importierten Moduls zugreifen.

```
In [ ]: # Importe schreibst du üblicherweise am Anfang deines Programms auf.
import time
print("Heute:", time.asctime())

# Durch den Schlüsselbegriff 'as' kannst du dem
# Import einen eigenen Namen geben.
import os.path as p
import os

# Gibt den aktuellen Pfad aus.
p.abspath(os.getcwd())

# Wenn du nur etwas Bestimmtes aus einer Datei
# importieren möchtest, kannst du das wie folgt tun:
from datetime import date

# Gibt den Wochentag (als Integer) eines vorgegebenen Datums zurück.
date(2021, 11, 2).weekday()

# Du kannst auch deine eigenen Dateien in dein Programm importieren.
# Wenn du eine Datei von einem Ordner in deinem aktuellen Verzeichnis
# importieren möchtest, gibst du den relativen Pfad durch Punkte getrennt an:
from resources.code.help_functions import sagHallo

sagHallo()
```

Fehlerbehandlung

Um zu verhindern, dass dein Programm bei einem Fehler während der Ausführung abstürzt, muss du festlegen, was im Fehlerfall passieren soll.

```
In [ ]: # Führe dieses Feld aus, ändere anschließend den Wert für 'y' und
# beobachte, was bei der Ausführung passiert.
x = 1
y = 0
ergebnis = 42

try:
    # Dieser Block wird ausgeführt.
    print("Beginn der Ausführung des try-Blocks...")
    ergebnis = x / y
except:
    # Wenn ein Fehler im try-Block auftritt,
    # wird dieser Block ausgeführt.
    print("Ups! Hier ist etwas schief gelaufen.")
finally:
    # Dieser Block ist optional, wird aber immer ausgeführt,
    # unabhängig davon, ob ein Fehler auftritt oder nicht.
    print("Der finally-Block wird immer ausgeführt.")

print(f"ergebnis = {ergebnis}")
```